11.3. AVL Trees

Keep the Tree Balanced, Keep the Search Fast

Previous Section:

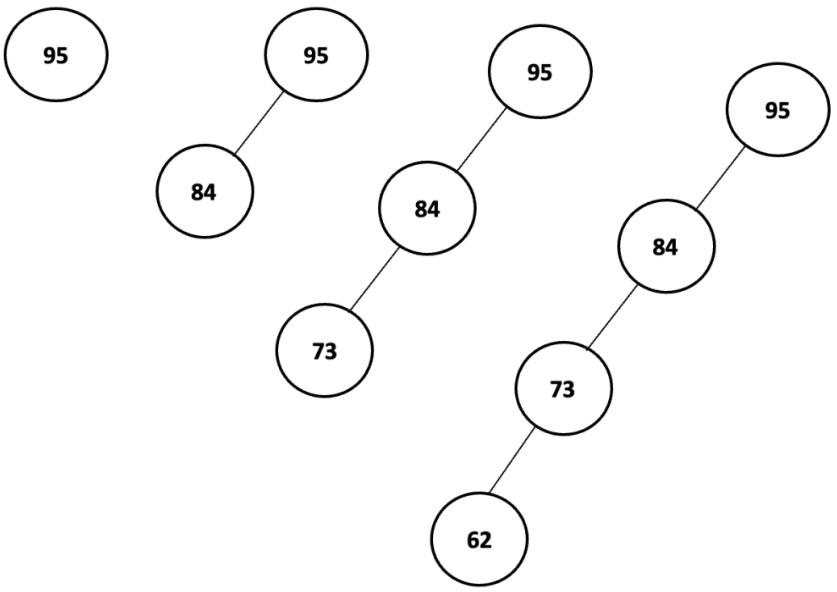
 11.2. Binary Search Trees

Contents:

When I first learned about the Binary Search Tree (BST), it seemed like the perfect structure. Searching, inserting, and deleting elements can all be performed efficiently using the BST property. However, BST has one major weakness.

Suppose we insert a sequence of values in decreasing order: `[95, 84, 73, 62]`

Or visualized as a tree:



Notice what happened. Instead of branching outward like a tree, every new node was inserted on the left side. The tree gradually lost its shape and became something very similar to a linked list.

The same problem occurs when inserting values in strictly increasing order: In both situations, the depth of the tree becomes equal to the number of nodes. For example, the above tree has $Depth = 4$ even if there are only 4 nodes at each depth

At this point, searching for an element is no longer efficient. Instead of repeatedly eliminating half of the search space like Binary Search, we may

need to traverse node after node until the desired value is found.

The complexity degrades from $O(logn)$ to $O(n)$. The reason is simple. Every insertion keeps pushing new nodes toward the leftmost or rightmost position, eventually transforming the BST into a linear chain.

To overcome this limitation, we need Binary Search Trees that can automatically maintain their shape. These trees are called **Self-Balancing Binary Search Trees**.

Some popular examples include: AVL Tree; Red-Black Tree; B-Tree

These structures perform rotations and rebalancing whenever necessary, preventing the tree from becoming degenerate and ensuring that operations remain close to $O(logn)$.

Among them, the AVL Tree is one of the earliest and most famous solutions.

1. Introduction to AVL Tree

AVL Tree is a self-balancing Binary Search Tree invented by **Adelson-Velsky** and **Landis**, from which the name **AVL** is derived.

The main idea is surprisingly simple: ***For every node in the tree, the heights of the left and right subtrees must not differ by more than 1.***

To measure this difference, we use something called the **Balance Factor (BF)**. The formula is:

$$\text{Balance Factor} = \text{Height(Left Subtree)} \text{ - } \text{Height(Right Subtree)}$$

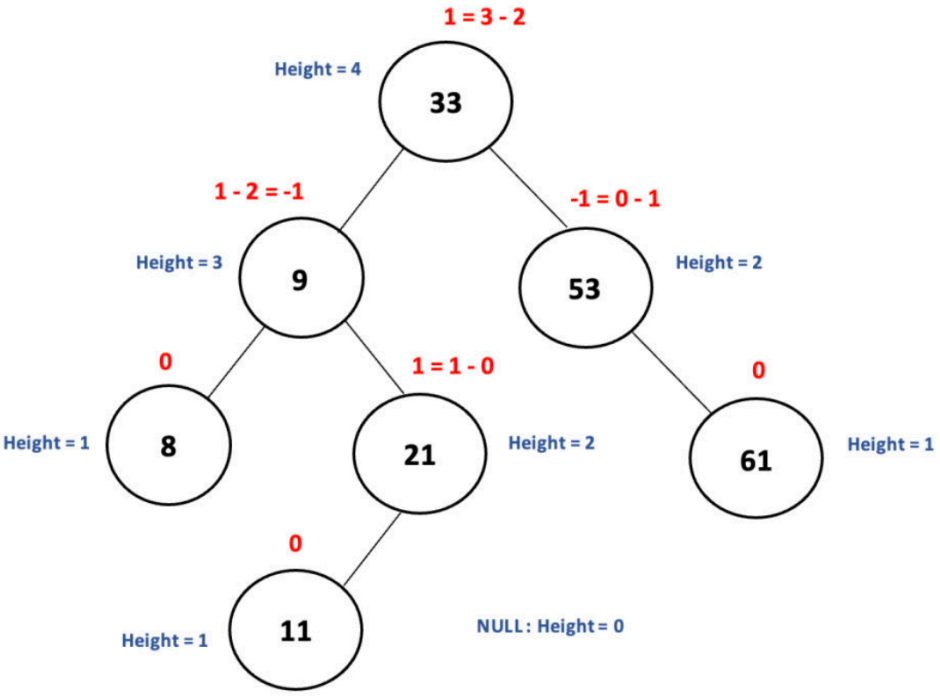
A node is considered balanced if its balance factor is either -1; 0; or 1. Any value outside this range means the node has become unbalanced and requires rotation.

1.1. AVL Tree Height Convention

One thing that often confuses beginners is that AVL Trees usually define height differently from ordinary Binary Trees.

- In many Binary Tree examples: $\text{Leaf Height} = 0$
- For AVL Trees: $\text{Empty Node (None)} = 0$; $\text{Leaf Node} = 1$

Consider the following tree:



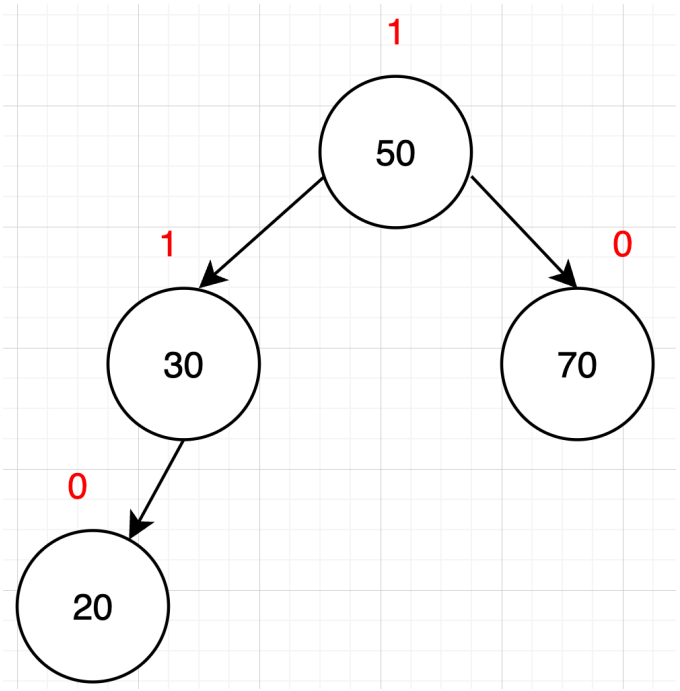
The red numbers represent the balance factor values

In general:

- **Rule 1 - Empty Node:** $Height(None) = 0$
- **Rule 2 - Leaf Node:** $Height(Leaf) = 1$
- **Rule 3 - Internal Node:** $Height(Node) = 1 + max(Height(LeftChild), Height(RightChild))$

1.2. Understanding Balance Factor

Let us compute the balance factor of a simple tree.



The numbers above the nodes represent the balance factor.

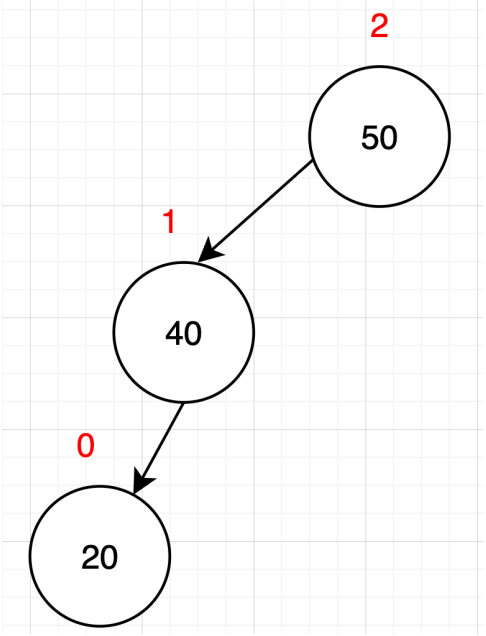
- **Step 1 - Heights:** $20 = 1; 70 = 1; 30 = 2; 50 = 3$
- **Step 2 - Balance Factors:**
  - Node 20: $BF = 0 - 0 = 0$
  - Node 70: $BF = 0 - 0 = 0$
  - Node 30: $BF = 1 - 0 = 1$
  - Node 50: $BF = 2 - 1 = 1$

Visualized:

Since every node has balance factor: $-1 \leq BF \leq 1$, the tree is balanced.

1.3. An Unbalanced Example

Now consider this tree:



The numbers above the nodes represent the balance factor.

- Heights: $30 = 1; 40 = 2; 50 = 3$
- Balance Factors:
  - Node 30: $BF = 0$
  - Node 40: $BF = 1$
  - Node 50: $BF = 2$

Notice that: $BF(50) = 2$. Since 2 is outside the valid range: $[-1, 0, 1]$, the node is unbalanced. An AVL Tree would immediately perform a rotation to restore balance.

1.4. Important Observation

When implementing AVL Trees in Python, we usually store only: `data, left child, right child, height` . For example:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
```

```
        self.right = None
        self.height = 1
```

Notice that we do **not** store the balance factor. Because the balance factor can always be calculated whenever needed. Therefore, storing height is sufficient.

When drawing AVL Trees on paper or explaining them in lectures, balance factors are often displayed because they make it easier to visualize which nodes are balanced and which nodes require rotation.

Finally, I can summarize the entire idea of an AVL Tree in a single sentence: **A Binary Search Tree is an AVL Tree if every node in the tree has a balance factor of $-1, 0,$ or 1. And whenever a node violates this rule, the AVL Tree automatically performs rotations to bring the tree back into balance.**

2. Basic Operations on AVL Tree

Just like a Binary Search Tree (BST), an AVL Tree supports the same fundamental operations: Traversal; Searching; Insertion; Deletion

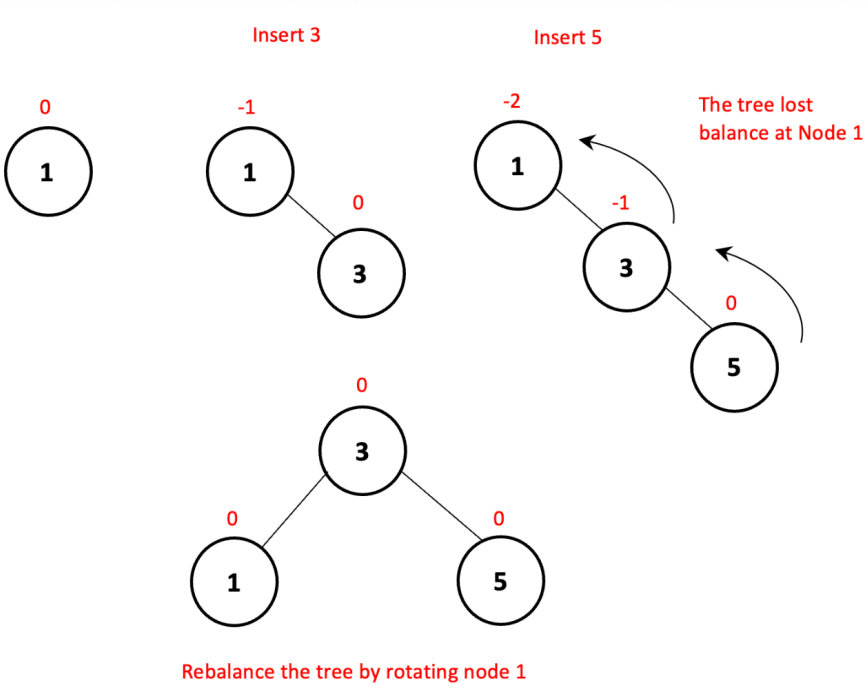
If I only look at traversal and searching, there is nothing particularly new. Both operations work exactly the same way as they do in a Binary Search Tree because an AVL Tree is still a Binary Search Tree at its core.

The interesting part begins when we perform **insertion or deletion**. When a new node is inserted, or an existing node is removed, the height of certain subtrees changes. This change may cause one or more nodes to become unbalanced. In other words, their balance factors may no longer remain within the valid range: $-1 \leq BF \leq 1$. When this happens, the AVL Tree must perform rotations to restore balance. This process is called **rebalancing**.

2.1. Insertion and Rebalancing

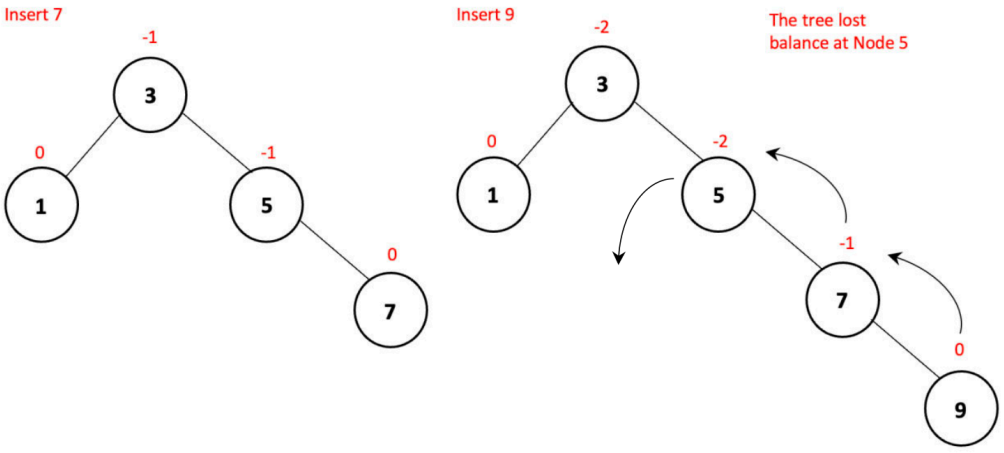
Suppose I insert a value into an AVL Tree. Initially, the insertion process is identical to a Binary Search Tree. We compare values and move left or right until I find an empty position.

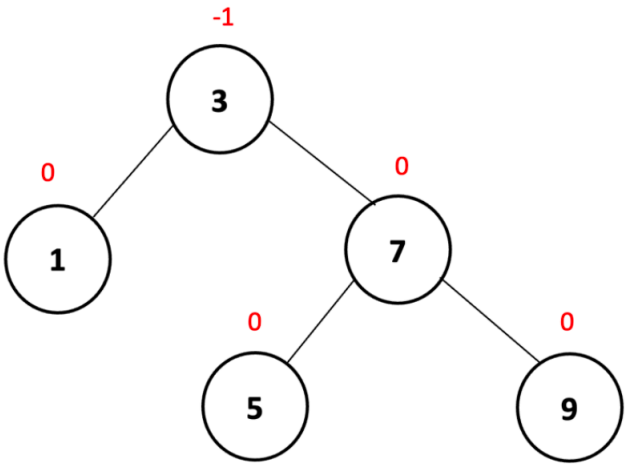
For example, inserting 3 and 5.



The insertion itself is valid according to BST rules. However, notice what happened to the heights. Node 1 now has: $BF = -2$. The tree becomes unbalanced. Therefore, the AVL Tree immediately performs a rotation to restore balance.

Another example, inserting 7 and 9

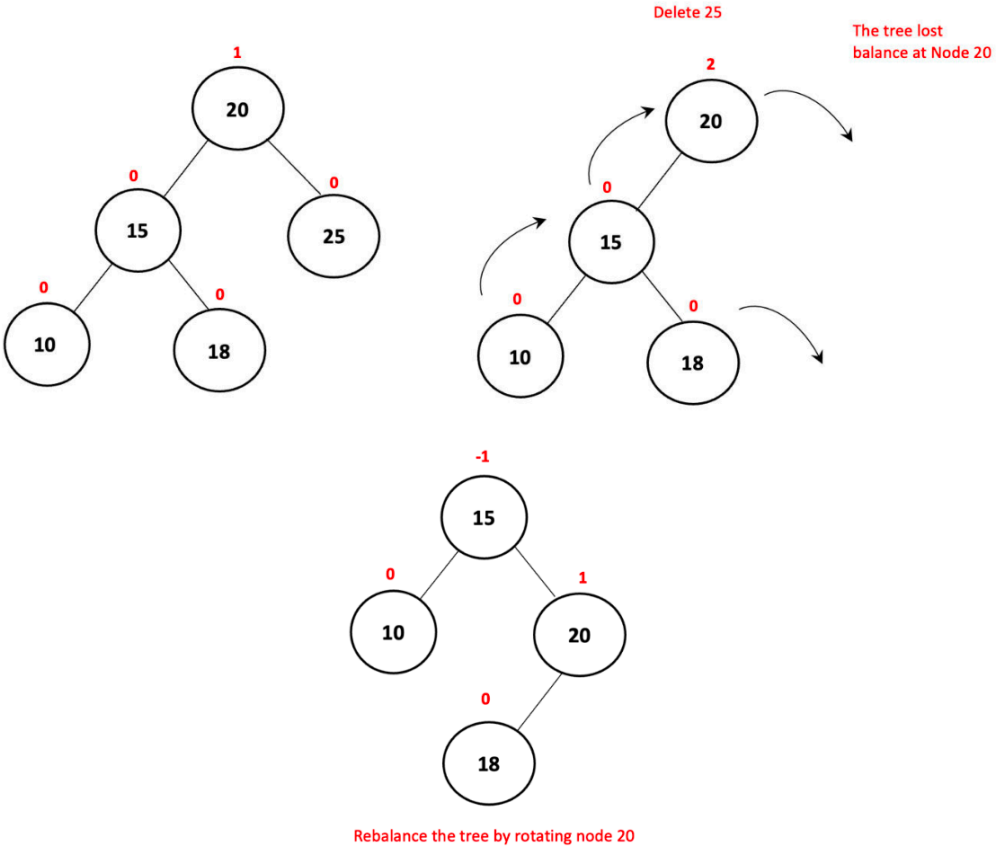




Rebalance the tree by rotating node 5

2.2. Deletion and Rebalancing

Deletion follows the same idea. The deletion process itself is identical to a Binary Search Tree. For example, suppose we remove a node.



Rebalance the tree by rotating node 20

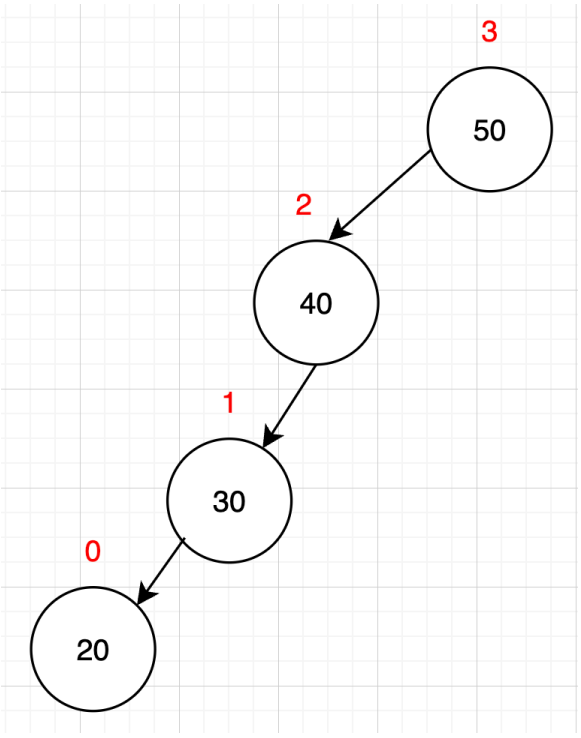
Again, the BST deletion is correct. However, removing a node decreases the height of a subtree. This change may cause the parent node to become

unbalanced. Therefore, after deletion, the AVL Tree checks balance factors and performs rotations whenever necessary.

2.3. Which Node Do We Rebalance?

A common question is: ***"If several nodes become unbalanced, which one should we rotate first?".*** The answer is surprisingly simple. After insertion or deletion, move upward from the modified position and look for the **first unbalanced ancestor**.

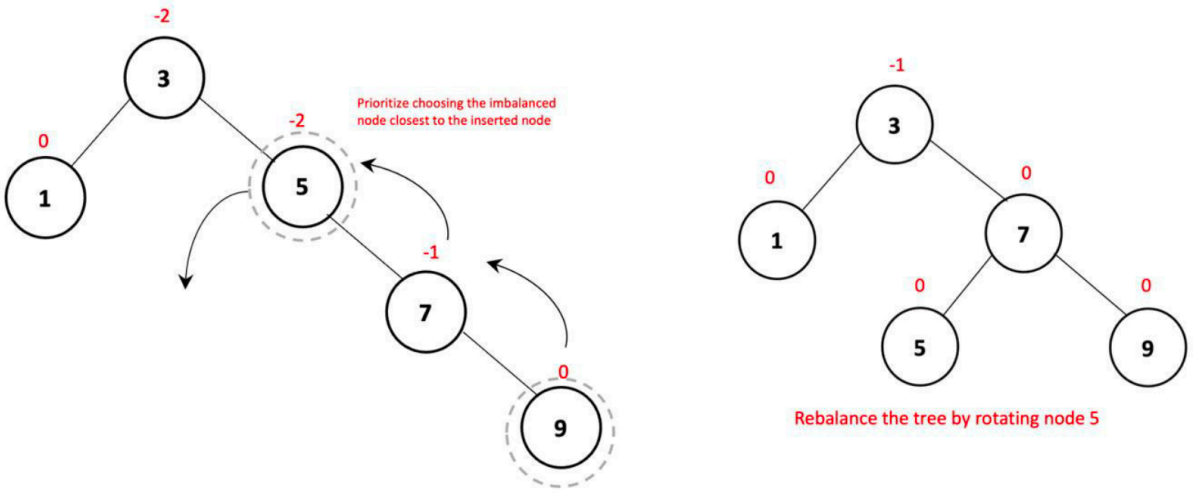
For example:



Suppose 20 was the newly inserted node. Starting from 20 and moving upward: `20 → 30 → 40 → 50`

The first node that violates the AVL condition is rotated. Once this subtree becomes balanced, the upper portions of the tree will automatically have correct heights as well.

Therefore, we only need to focus on the nearest unbalanced node when moving upward from the insertion or deletion position.



Suppose we insert 9

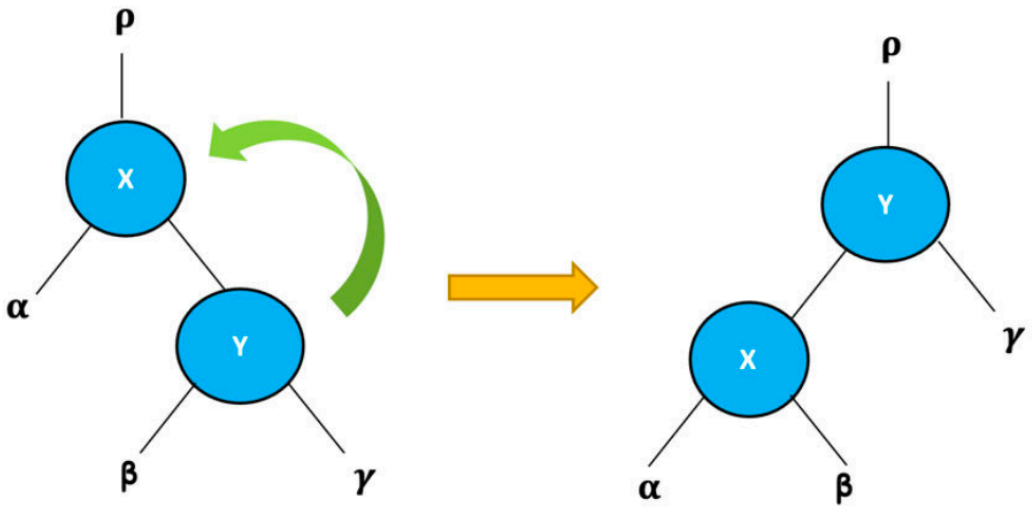
2.4. Rotation Operations

The AVL Tree uses four rotation operations to restore balance. These rotations are local operations. They only rearrange a small portion of the tree while preserving the Binary Search Tree property.

The four rotations are: **Left Rotation (RR Rotation); Right Rotation (LL Rotation); Left-Right Rotation (LR Rotation); Right-Left Rotation (RL Rotation)**

- **Left Rotation (RR Rotation):** A Left Rotation is performed when the tree becomes heavily tilted to the right.

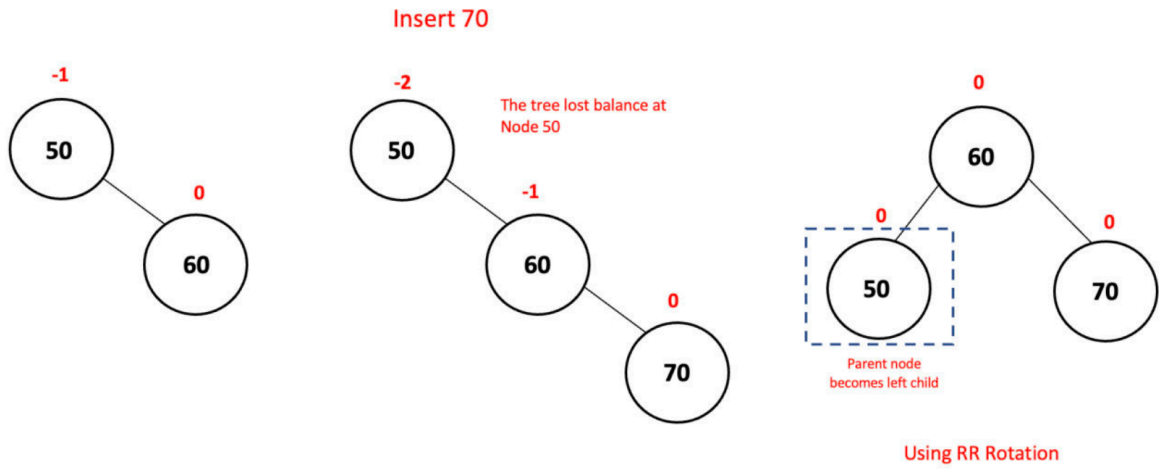
The newly inserted node γ is located inside the right subtree of the right child. This causes: $BF(x) < -1$. To restore balance, rotate x counterclockwise.



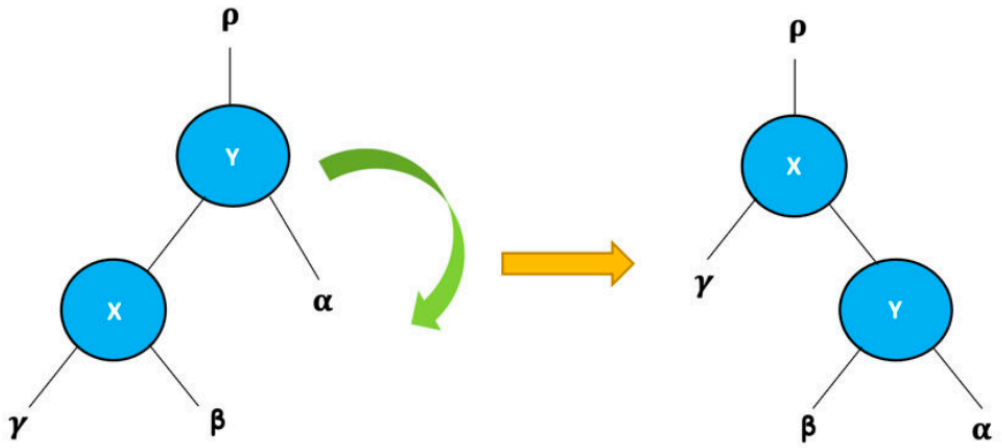
Implementation idea:

```
y = x.right
β = y.left

y.left = x
x.right = β


Where:
x = node being rotated
y = new root
β = subtree that changes position
```

Example:



- **Right Rotation (LL Rotation):** A Right Rotation is the mirror image of Left Rotation. It is used when the tree becomes heavily tilted to the left.

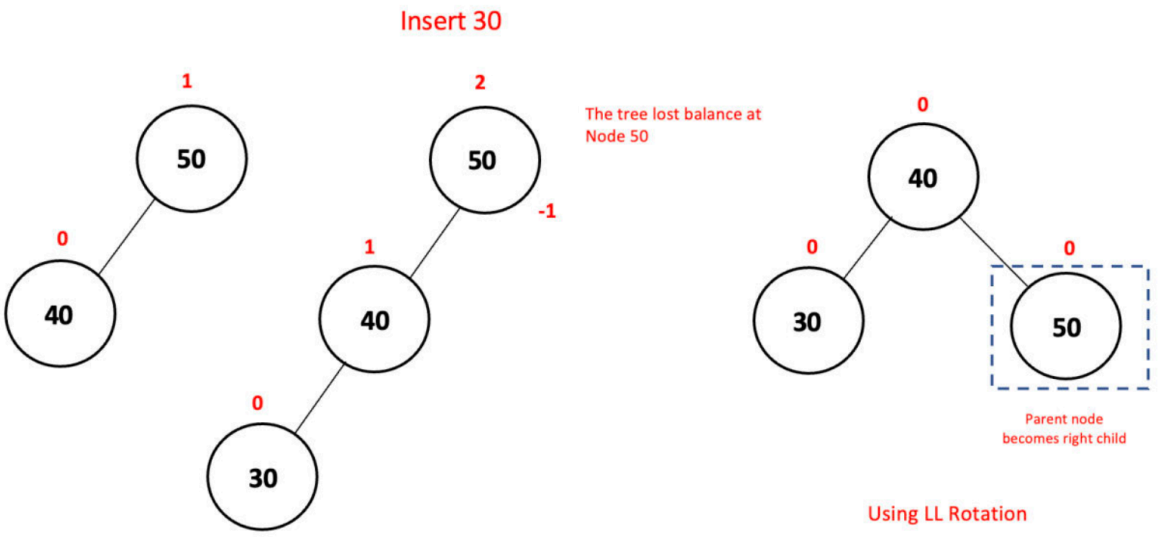


The newly inserted node γ belongs to the left subtree of the left child. This causes: BF(y) > 1. So we rotate y-clockwise.

Implementation idea:

```
x = y.left
β = x.right

x.right = y
y.left = β

Where:
y = node being rotated
x = new root
β = subtree that changes position
```

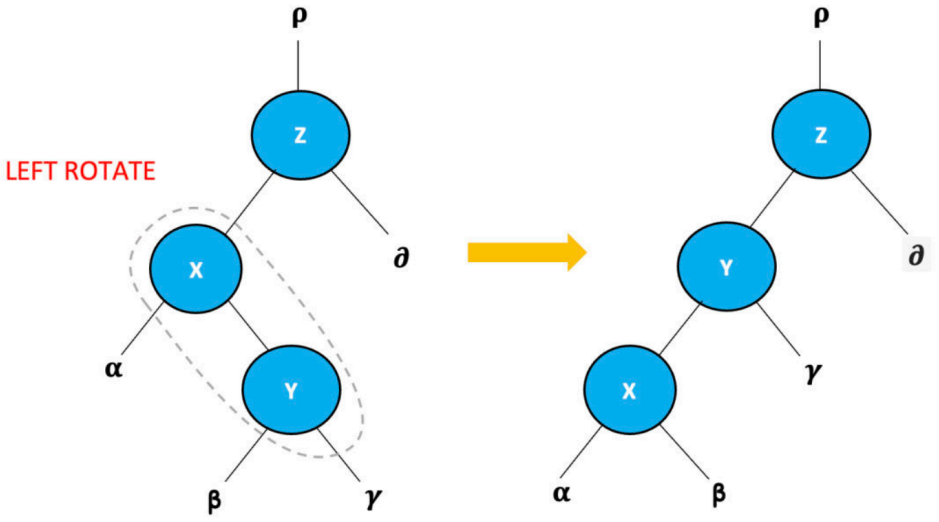
Example:



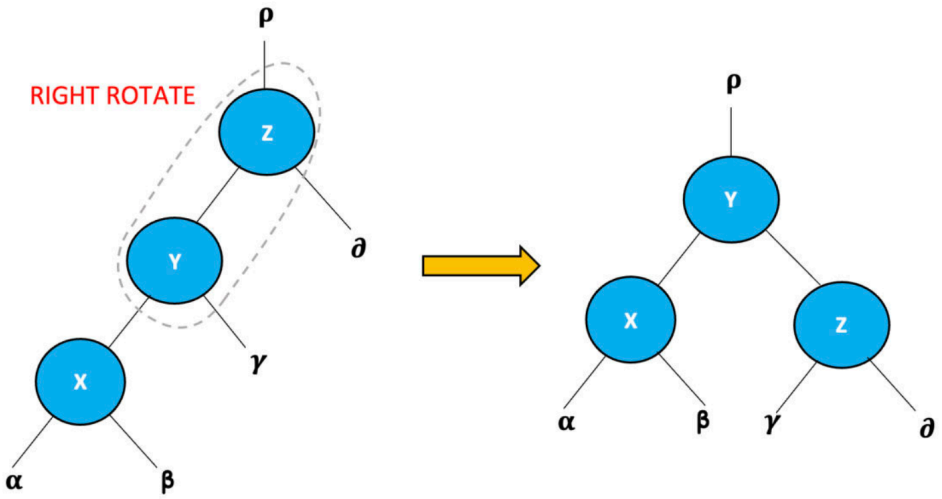
- **Left-Right Rotation (LR Rotation):** Sometimes the tree leans left, but not because of the left child itself. Instead, the imbalance comes from the right subtree of the left child.

The tree is left-heavy, but the insertion happened on the right side of x. A single Right Rotation is not enough. Therefore, we perform two rotations.

Step 1 - Left Rotation: Rotate x and y.

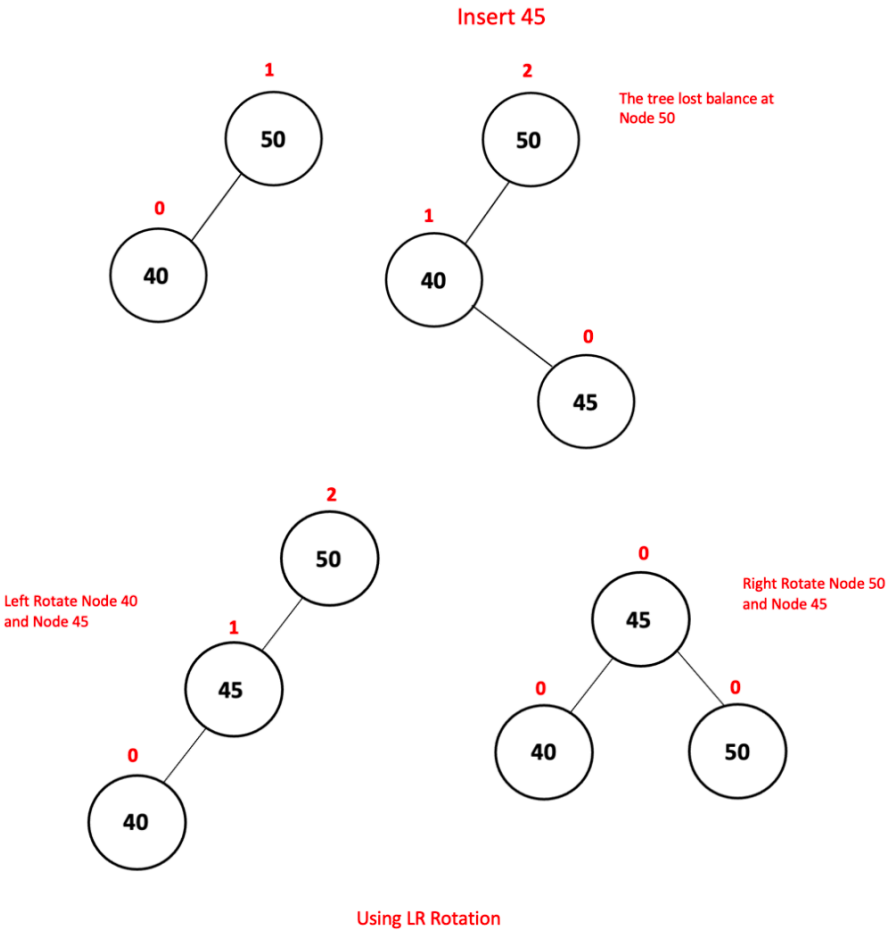


Step 2 - Right Rotation: Rotate z and y.



The tree becomes balanced again.

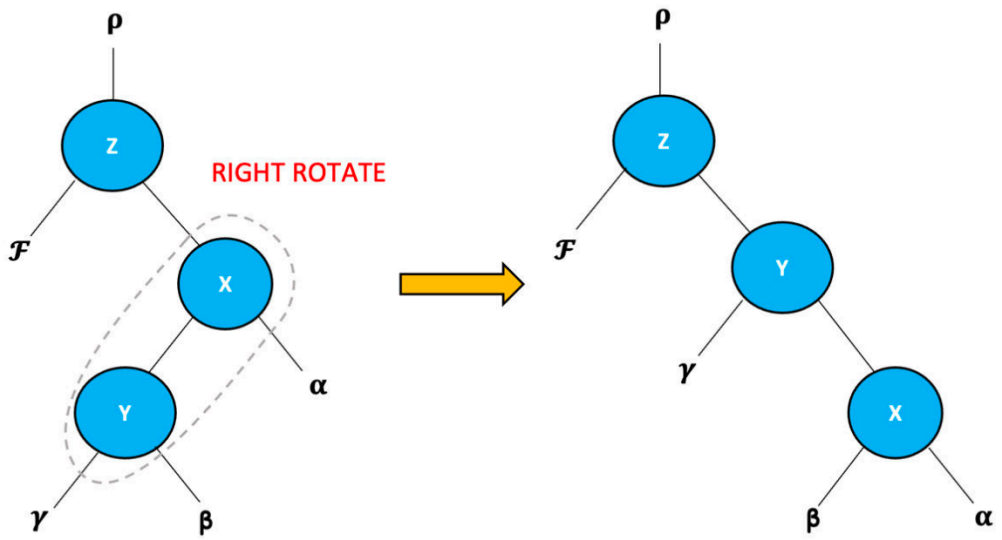
Example:



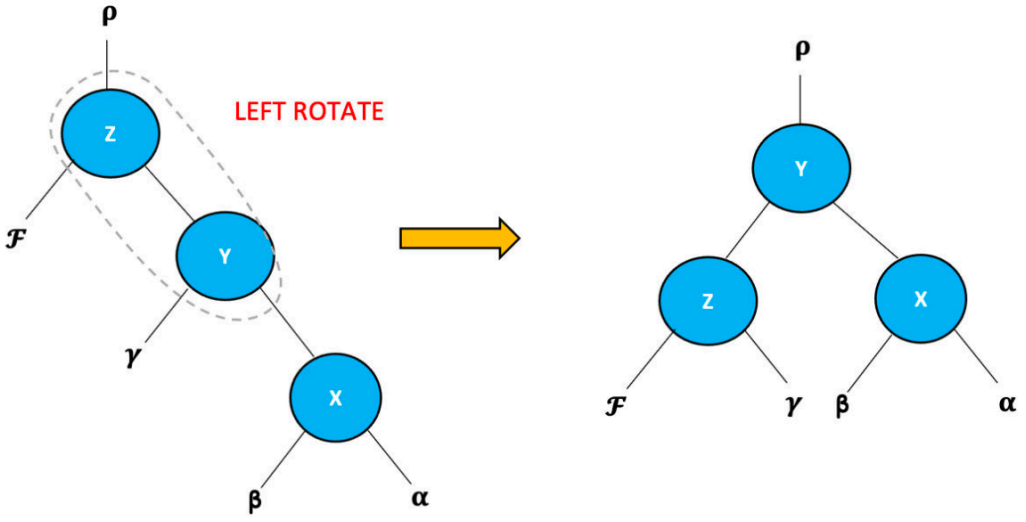
- **Right-Left Rotation (RL Rotation):** This case is the mirror image of LR Rotation.

The tree is right-heavy, but the insertion happened on the left side of x. Again, a single rotation cannot solve the problem.

Step 1 - Right Rotation: Rotate x and y.

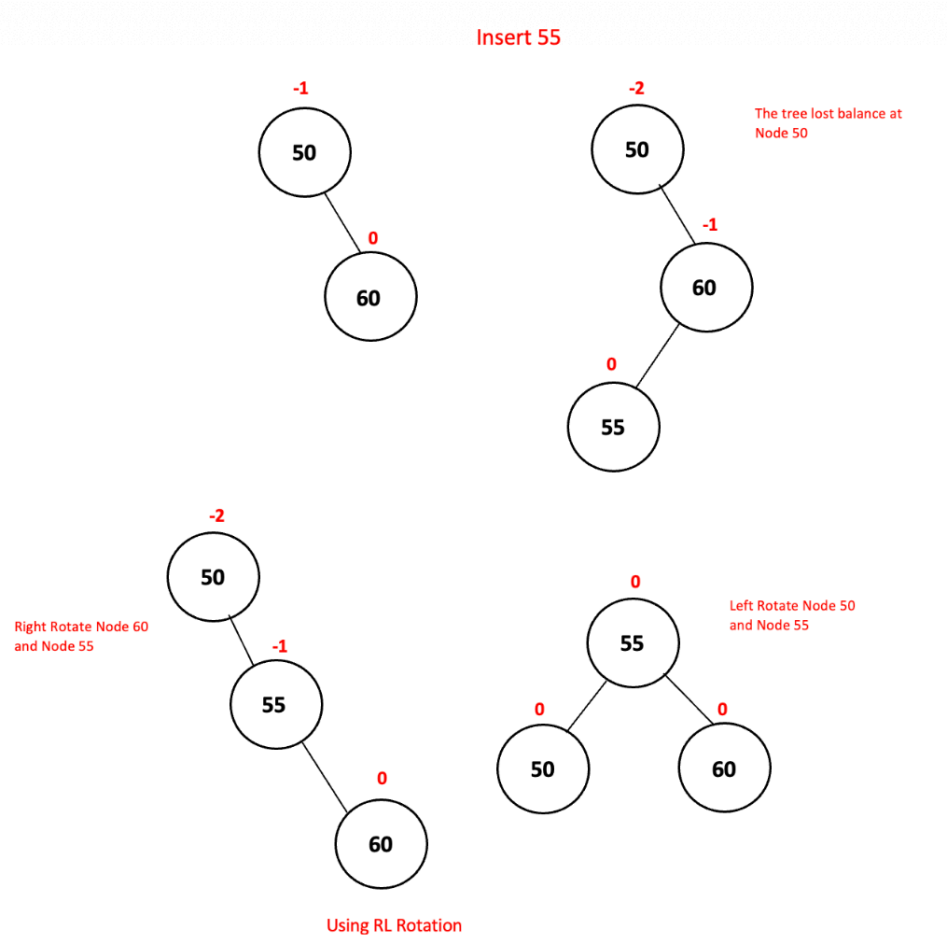


Step 2 - Left Rotation: Rotate z and y.



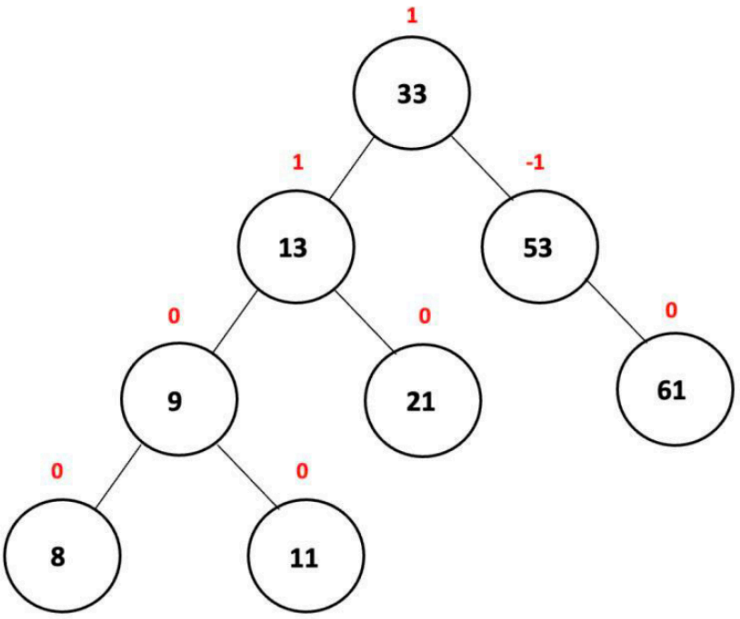
Balance is restored.

Example:



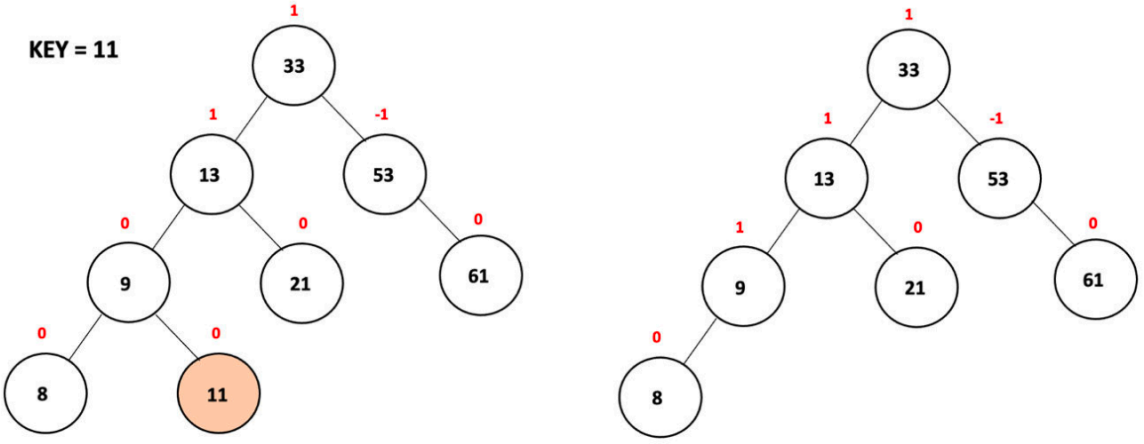
2.5. AVL Tree Deletion Cases

Deletion in AVL Tree follows exactly the same three cases as Binary Search Tree.



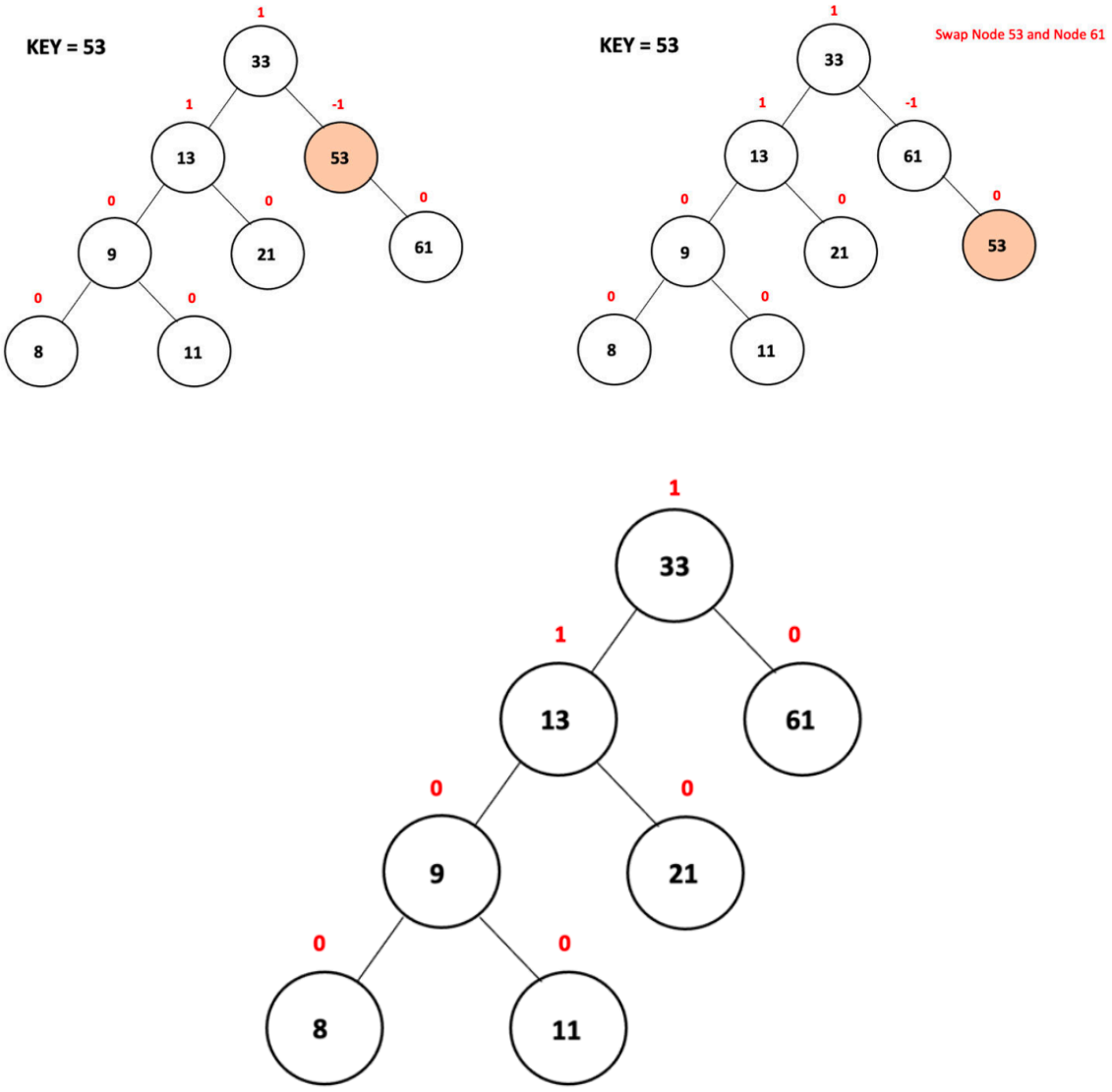
- **Case 1: Deleting a Leaf Node**

This is the simplest case. After deletion, heights and balance factors are updated. If any ancestor becomes unbalanced, rotations are performed.



- **Case 2: Deleting a Node with One Child**

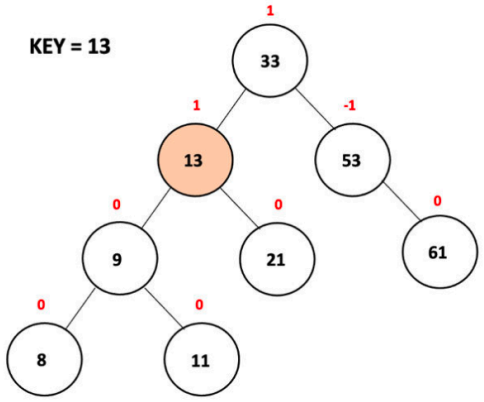
The child replaces the deleted node. Then heights and balance factors are updated. Rotations may be required afterward.

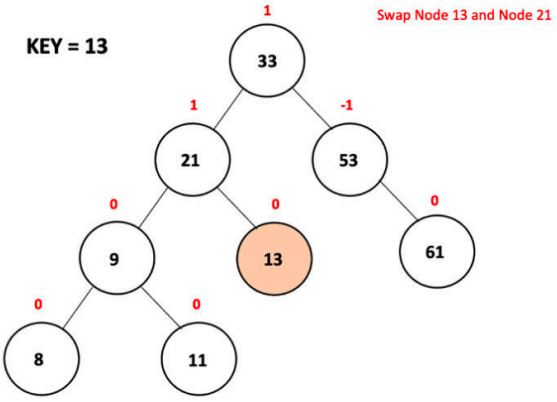


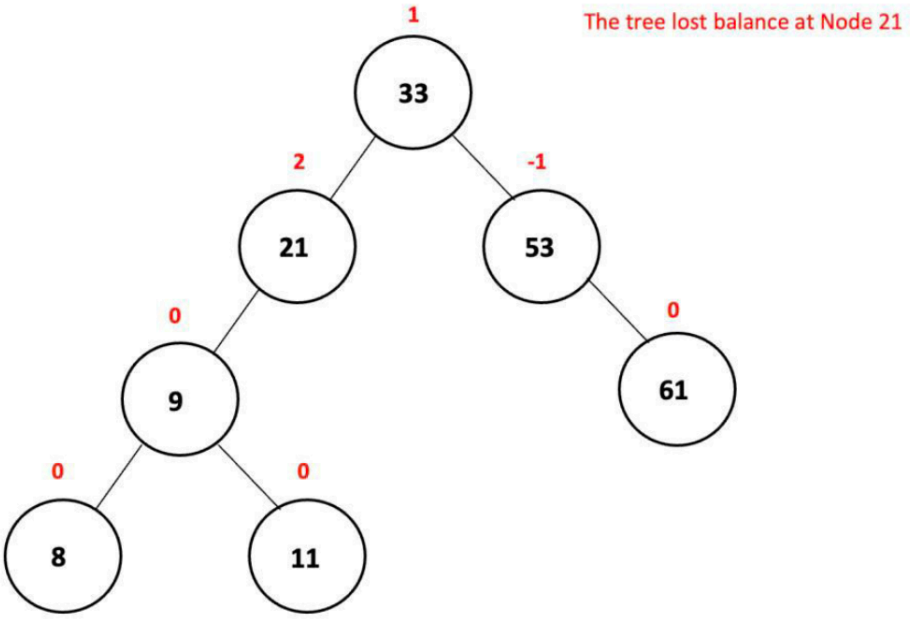
- **Case 3: Deleting a Node with Two Children**

Suppose we want to delete 13.

We first find its in-order successor: 21. Then substitute 21 with 13. Finally, remove the original 13 node and update heights. Again, rotations may be necessary.





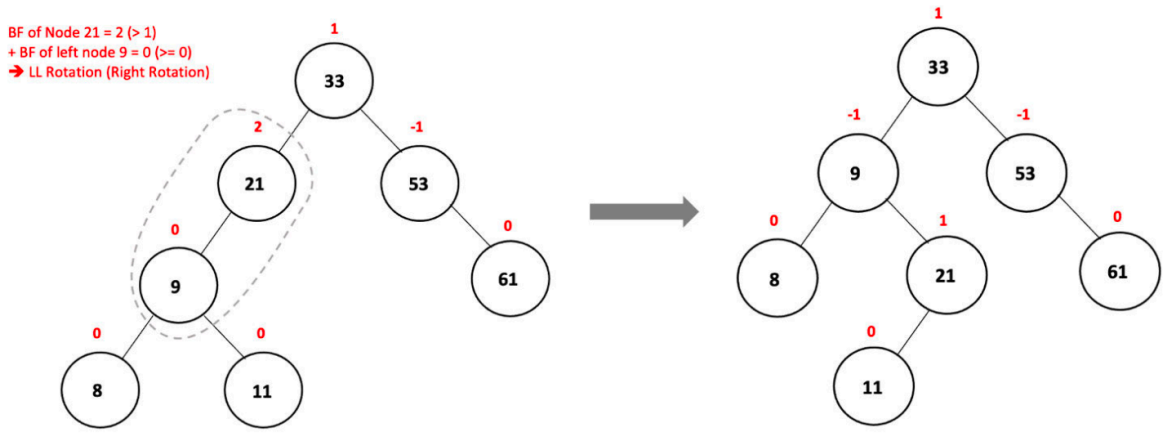


2.6. Choosing the Correct Rotation

After insertion or deletion, 21 check the balance factor of the current node.

- **Left Heavy:** $BF > 1$ → The tree leans left.
  - **Left Child Also Leans Left:** $\mathrm{BF}(\text{left child}) \geq 0$ → Perform: LL Rotation (Right Rotation)
  - **Left Child Leans Right:** $\mathrm{BF}(\text{left child}) < 0$ → Perform: LR Rotation
- **Right Heavy:** $BF < -1$ → The tree leans right.

- **Right Child Also Leans Right:** $\mathrm{BF}(\text{right child}) \leq 0$ → Perform: RR Rotation (Left Rotation)
- **Right Child Leans Left:** $\mathrm{BF}(\text{right child}) > 0$ → Perform: RL Rotation



In this case, the tree leans left, and left child also leans left → LL Rotation

2.7. Time Complexity

One of the greatest strengths of AVL Tree is that rotations are extremely cheap. Every rotation only rearranges a few pointers. Therefore:

- Rotation $= O(1)$
- Traversal still requires visiting every node → Traversal $= O(n)$.
- Because an AVL Tree continuously maintains balance, its height remains approximately logarithmic. As a result:
  - Searching $= O(logn)$
  - Insertion $= O(logn)$
  - Deletion $= O(logn)$

This is the major advantage of AVL Trees over ordinary Binary Search Trees. When I insert nodes into a normal BST, the structure may eventually become:

```
10
 \
 20
   \
   30
```

```
    \
    40
```

Which behaves like a linked list.

However, every insertion into an AVL Tree is followed by rebalancing whenever necessary. Therefore, the tree remains close to:

```
      20
     /  \
   10    30
           \
           40
```

Or another balanced form.

The same principle applies to deletion. Even in the worst case, the AVL Tree avoids degenerating into a linear structure, allowing searching, insertion, and deletion to remain efficient throughout the lifetime of the tree.

3. AVL Tree with Python

Before implementing the AVL Tree itself, we first need a way to perform Breadth-First Traversal (Level Order Traversal). Since Breadth-First Traversal requires a queue, I will use a custom Circular Queue implementation.

3.1. Circular Queue Implementation

The AVL Tree traversal in this implementation uses a Circular Queue instead of Python's built-in queue structures. The purpose of this queue is to temporarily store nodes waiting to be visited during Breadth-First Traversal.

Complete Circular Queue Class

```python
class CircularQueue:
    def __init__(self, maximum=100):
        self.maximum = maximum
        self.front = -1
        self.rear = -1
        self.items = [None] * maximum
```

```
    def is_empty(self):
        return self.front == -1

    def remove(self):
        if self.front == -1:
            return False

        if self.front == self.rear:
            self.front = -1
            self.rear = -1
        else:
            if self.front == self.maximum - 1:
                self.front = 0
            else:
                self.front += 1
        return True

    def enqueue(self, data):
        if (self.front == 0 and self.rear == self.maximum -
1) or \
           (self.front == self.rear + 1):
            print("Queue Overflow")
            return False

        if self.front == -1:
            self.front = 0
            self.rear = 0
        else:
            if self.rear == self.maximum - 1:
                self.rear = 0
            else:
                self.rear += 1

        self.items[self.rear] = data
        return True

    def dequeue(self):
```

```
        if self.front != -1:
            temp = self.items[self.front]
            self.remove()
            return temp
        else:
            print("Queue is empty")
            return False

    def traverse(self):
        front_index = self.front
        rear_index = self.rear

        if front_index == -1:
            print("Queue is empty")
            return

        if front_index <= rear_index:
            while front_index <= rear_index:
                print(f"{self.items[front_index]} <-", end
=" ")
                front_index += 1
        else:
            while front_index <= self.maximum - 1:
                print(f"{self.items[front_index]} <-", end
=" ")
                front_index += 1

            front_index = 0

            while front_index <= rear_index:
                print(f"{self.items[front_index]} <-", end
=" ")
                front_index += 1

        print()
```

How the Circular Queue Works

The queue stores elements in a fixed-size list: `self.items = [None] * maximum`. Instead of continuously shifting elements after removal, two pointers are maintained: `front` and `rear`. For example:

```
Index : 0   1   2   3   4

Queue : A   B   C
        ↑       ↑
      front   rear

When an element is removed:

Index : 0   1   2   3   4

Queue : _   B   C
            ↑   ↑
          front rear
```

The queue simply moves the front pointer forward rather than shifting every element.

When the rear reaches the last index:

```
Index : 0   1   2   3   4

Queue : _   B   C   D   E
                    ↑   ↑
                  front rear

The next insertion wraps around to index 0:

Index : 0   1   2   3   4

Queue : F   B   C   D   E
        ↑           ↑
      rear        front
```

This wrapping behavior is why it is called a Circular Queue.

3.2. AVL Tree Node Structure

Every node stores four pieces of information:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1

    def __str__(self):
        return f"[{self.data}]"

    def display(self):
        if self.data is not None:
            print(f"{self.data} [h={self.height}]", end=",
")
```

Unlike a normal Binary Search Tree node, an AVL node contains an additional attribute: `self.height` . This value is required for calculating balance factors.

For example:

```
    50(h=3)
    /     \
30(h=2)  70(h=1)
```

Whenever insertion or deletion occurs, these heights are updated automatically.

From here, we start create our AVL Tree class:

```
class AVLTree:
    def __init__(self):
        self.root = None
```

3.3 Breadth-First Traversal

To display the AVL Tree, Breadth-First Traversal is used.

- **Traversal Function**

```
def traverse(self):
    self.breadth_first_traverse(self.root)
```

The actual traversal is performed by:

```
def breadth_first_traverse(self, root_node):
    current = root_node

    if current is None:
        print("Tree is empty")
        return

    nodes_queue = CircularQueue()
    nodes_queue.enqueue(current)

    while not nodes_queue.is_empty():
        node = nodes_queue.dequeue()

        self.process_node(node)

        if node.left is not None:
            nodes_queue.enqueue(node.left)
        if node.right is not None:
            nodes_queue.enqueue(node.right)
```

Consider:

```
      50
     /  \
    30   70
   / \
  20 40
```

The queue evolves as follows:

```
[50]

- Visit 50: [30,70]
- Visit 30: [70,20,40]
- Visit 70: [20,40]
```

```
- Visit 20: [40]
- Visit 40: []

Output: 50, 30, 70, 20, 40
```

3.4. Insertion in AVL Tree

Insertion starts exactly like a Binary Search Tree.

- **Public Insert Function**

```
def insert(self, data):
    self.root = self.insert_tree(self.root, data)
```

- **Recursive Insertion**

```
def insert_tree(self, current, data):

    if current is None:
        return Node(data)

    if data < current.data:
        current.left = self.insert_tree(current.left, data)

    elif data > current.data:
        current.right = self.insert_tree(current.right, dat
a)

    else:
        print("Node exists:", data)
        return current
```

After insertion, the height must be updated.

```
current.height = 1 + max(self.get_height(current.left),
                         self.get_height(current.righ
t))
```

Next, the balance factor is computed.

```
balance = self.get_balance_factor(current)
```

If the balance factor exceeds the AVL limits, rotations are performed.

- **LL Rotation Case**

```
Example insertion: 30, 20, 10

Before balancing:

      30
     /
    20
   /
  10
```

Balance Factor: $BF(30) = 2$. AVL performs:

```
return self.right_rotate(current)
```

Result:

```
     20
    /  \
  10    30
```

- **RR Rotation Case**

```
Example:
Insert: 10, 20, 30

Before balancing:

10
 \
 20
   \
   30
```

AVL performs:

```
return self.left_rotate(current)
```

Result:

```
    20
   /  \
  10   30
```

- **LR Rotation Case**

```
Example:

Insert: 30, 10, 20

Before balancing:

    30
   /
  10
    \
     20
```

AVL performs:

```
current.left = self.left_rotate(current.left)
return self.right_rotate(current)
```

Result:

```
    20
   /  \
  10   30
```

- **RL Rotation Case**

```
Example:
Insert: 10, 30, 20

Before balancing:
10
  \
   30
  /
20
```

AVL performs:

```
current.right = self.right_rotate(current.right)
return self.left_rotate(current)
```

Result:

```
    20
   /  \
  10   30
```

3.5. Deletion in AVL Tree

Deletion also begins exactly like a Binary Search Tree.

- **Public Delete Function**

```
def delete(self, data):
    self.root = self.delete_tree(self.root, data)
```

- **Recursive Delete Function**

```
def delete_tree(self, current, data):
```

The algorithm first locates the target node.

- **Case 1 - Leaf Node:** The node is removed immediately.

```
Before:

      50
     /
   30

Delete: 30
After:
  50
```

- **Case 2 - One Child:** The child replaces the deleted node.

```
Before:

      50
     /
   30
   /
 20

Delete: 30
After:

    50
    /
   20
```

- **Case 3 - Two Children:** The original successor node is then deleted.

```
Before:

        50
       /  \
     30    70
          /
        60

Delete: 50
The successor is: 60
After replacement:

        60
       /  \
     30    70
```

Rebalancing After Deletion

Once deletion finishes, the balance factor is checked again.

```
balance = self.get_balance_factor(current)
```

- If `balance > 1` → the tree leans left.
- If `balance < -1` → the tree leans right.

The same four AVL rotations are then applied: LL Rotation; RR Rotation; LR Rotation; RL Rotation

This ensures the tree remains balanced after every deletion.

3.6. Rotation Functions

The most important part of AVL Trees is the rotation mechanism.

- **Left Rotation**

```
def left_rotate(self, x):

    y = x.right
    beta = y.left
    y.left = x
    x.right = beta

    x.height = 1 + max(self.get_height(x.left),
                       self.get_height(x.right))
    y.height = 1 + max(self.get_height(y.left),
                       self.get_height(y.right))
    return y
```

Visualization:

```
Before:

   x
    \
```

```
      y
     /
   beta

After:

      y
     /
    x
     \
    beta
```

- **Right Rotation**

```python
def right_rotate(self, y):
    x = y.left
    beta = x.right
    x.right = y
    y.left = beta

    y.height = 1 + max(self.get_height(y.left),
                       self.get_height(y.right))
    x.height = 1 + max(self.get_height(x.left),
                       self.get_height(x.right))

    return x
```

Visualization:

```
Before:

       y
      /
     x
      \
     beta

After:

     x
      \
       y
```

```
      /
   beta
```

Below is the full implementation of the AVL Tree class:

avl_tree.py

AVL Tree Visualization

https://www.cs.usfca.edu/~galles/visualization/AVLtree.html

Summary

AVL Tree is a self-balancing Binary Search Tree that maintains the balance factor of every node between -1 and 1.

Unlike a normal BST, AVL Trees automatically perform rotations whenever insertion or deletion causes imbalance. These rotations include: LL Rotation (Right Rotation); RR Rotation (Left Rotation); LR Rotation; RL Rotation

Because the tree continuously maintains a balanced structure, its height remains approximately logarithmic. As a result:

Traversal	$O(n)$
Searching	$O(logn)$
Insertion	$O(logn)$
Deletion	$O(logn)$
Rotation	$O(1)$

This makes AVL Trees significantly more efficient than ordinary Binary Search Trees in situations where frequent searching, insertion, and deletion operations are performed.

References

https://drive.google.com/file/d/1tOqiD1hwN4q_ZbZHeOp-e3oeJfgTEdDq/view?usp=drive_link

https://www.geeksforgeeks.org/dsa/introduction-to-avl-tree/

https://www.programiz.com/dsa/avl-tree

https://www.youtube.com/watch?v=jDM6_TnYIqE

Next Section:

11.4. Red-Black Trees